

Introduction to NumPy

Arrays, Vectorization, Broadcasting, and Numerical Computing

Duc-Minh Vu

SLSCM Lab – FDA – NEU

August 3, 2026

Lesson Roadmap

- 1 What NumPy is
- 2 Why NumPy is useful
- 3 Installation and importing
- 4 Creating arrays
- 5 Array properties
- 6 Indexing and slicing
- 7 Reshaping, resizing, stacking, splitting
- 8 Broadcasting
- 9 Arithmetic and aggregation
- 10 Universal functions
- 11 Boolean operations
- 12 Linear algebra
- 13 Random numbers and statistics
- 14 Integration and common errors

Learning Outcomes

By the end of this lesson, learners will be able to:

- ▶ Explain the purpose of NumPy in numerical computing.
- ▶ Distinguish NumPy arrays from Python lists.
- ▶ Create one-dimensional and multidimensional arrays.
- ▶ Inspect `shape`, `size`, `ndim`, and `dtype`.
- ▶ Use indexing, slicing, reshaping, stacking, and splitting.
- ▶ Apply vectorized arithmetic, broadcasting, aggregation, and universal functions.
- ▶ Perform basic linear algebra and descriptive statistics.
- ▶ Generate random values and understand reproducibility.
- ▶ Explain how NumPy connects with Pandas, SciPy, and Scikit-learn.

Prerequisites and Setup

Recommended background

- ▶ Basic Python variables, lists, loops, and functions.
- ▶ A working Python environment: Jupyter Notebook, JupyterLab, Google Colab, or similar.

Installation and import

```
pip install numpy
```

```
import numpy as np  
print(np.__version__)
```

What Is NumPy?

NumPy, short for **Numerical Python**, is a core Python library for fast numerical computing. Its central object is the **N-dimensional array**, called `ndarray`. A NumPy array can represent:

- ▶ A one-dimensional vector.
- ▶ A two-dimensional matrix.
- ▶ A three-dimensional tensor.
- ▶ Higher-dimensional numerical structures.

Why it matters

NumPy stores homogeneous data efficiently and supports fast operations over whole arrays.

NumPy Provides

- ▶ Fast array operations.
- ▶ Vectorized calculations.
- ▶ Broadcasting.
- ▶ Linear algebra functions.
- ▶ Statistical functions.
- ▶ Random-number generation.
- ▶ Integration with Pandas, SciPy, Matplotlib, and Scikit-learn.

Main Features

Feature	Meaning
ndarray	Efficient multidimensional array with one common data type.
Vectorization	Apply calculations to entire arrays without explicit Python loops.
Broadcasting	Operate on compatible arrays with different shapes.
Linear algebra	Matrix multiplication, determinants, inverses, eigenvalues, and vector products.
Statistics	Mean, median, variance, standard deviation, percentiles, and quantiles.
Integration	Works with Pandas, SciPy, Matplotlib, Scikit-learn, and Statsmodels.

Quick Check: What Is NumPy?

Question 1. What is the main data structure in NumPy?

- ▶ A. DataFrame
- ▶ B. ndarray
- ▶ C. dictionary
- ▶ D. tuple

Question 2. True or false? NumPy is designed mainly for numerical computing.

Why Learn NumPy?

NumPy provides an efficient foundation for numerical work in Python.

- ▶ Executes vectorized operations much faster than many ordinary Python loops.
- ▶ Stores homogeneous numerical data more compactly than standard Python lists.
- ▶ Provides optimized functions for matrices, linear algebra, and transformations.
- ▶ Supports random-number generation and statistical analysis.
- ▶ Expresses complex mathematical operations using concise syntax.
- ▶ Serves as the numerical foundation for many data-science libraries.

Python List vs. NumPy Array

Python list: mixed objects

```
values = [10, 2.5, "Python", True]
```

- ▶ Flexible object container.
- ▶ Not optimized for large numerical arrays.

NumPy array: homogeneous values

```
import numpy as np  
values = np.array([10, 20, 30, 40])
```

- ▶ Regular structure.
- ▶ Efficient numerical computation.

Example: Multiplying Values

Python list

```
values = [1, 2, 3, 4]
result = []

for value in values:
    result.append(value * 10)

print(result)
# [10, 20, 30, 40]
```

NumPy vectorization

```
import numpy as np
values = np.array([1, 2, 3, 4])

result = values * 10
print(result)
# [10 20 30 40]
```

Key idea

The same operation is applied to all elements at once.

Quick Check: Why Learn NumPy?

Question 1. Why are NumPy arrays efficient for numerical calculations?

- ▶ A. They always contain text
- ▶ B. They use a homogeneous and optimized array structure
- ▶ C. They do not use memory
- ▶ D. They automatically connect to the internet

Question 2. Which expression multiplies every element of a NumPy array `a` by 10?

- ▶ A. `a * 10`
- ▶ B. `a.append(10)`
- ▶ C. `a.add("10")`
- ▶ D. `a.sort(10)`

Creating NumPy Arrays

NumPy arrays can be created from Python lists, tuples, ranges, or built-in NumPy functions.

```
import numpy as np

a = [9, 3, 3, 5]
arr = np.array(a)
print(arr)
# [9 3 3 5]
```

```
arr = np.array([10, 20, 30, 40])
print(arr)
```

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print(matrix)
```

Multidimensional Arrays

A three-dimensional array can represent stacked matrices or tensors.

```
tensor = np.array([
    [
        [1, 2],
        [3, 4]
    ],
    [
        [5, 6],
        [7, 8]
    ]
])
print(tensor)
```

Interpretation

Dimensions often correspond to observations, features, time, channels, or scenarios.

Common Array-Creation Functions

```
np.zeros(5)
np.zeros((2, 3))
np.ones((2, 3))
np.full((2, 3), 7)
```

```
np.arange(0, 10, 2)
# [0 2 4 6 8]

np.linspace(0, 1, 5)
# [0.    0.25 0.5  0.75 1.]

np.eye(3)
```

Quick Check: Creating Arrays

Question 1. Which function converts a Python list into a NumPy array?

- ▶ A. `np.array()`
- ▶ B. `np.list()`
- ▶ C. `np.convert()`
- ▶ D. `np.ndarray_list()`

Question 2. Which function creates evenly spaced values between two endpoints?

- ▶ A. `np.linspace()`
- ▶ B. `np.stack()`
- ▶ C. `np.split()`
- ▶ D. `np.mean()`

Array Properties

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(arr.ndim)          # 2
print(arr.shape)         # (2, 3)
print(arr.size)          # 6
print(arr.dtype)
print(arr.itemsize)
```

Attribute	Meaning
ndim	Number of dimensions.
shape	Size along each dimension.
size	Total number of elements.
dtype	Data type.
itemsize	Number of bytes used by each element.

Quick Check: Array Properties

Question 1. Which attribute returns the dimensions of an array?

- ▶ A. shape
- ▶ B. mean
- ▶ C. append
- ▶ D. index

Question 2. Which attribute returns the total number of elements?

- ▶ A. size
- ▶ B. dtype
- ▶ C. ndim
- ▶ D. itemsize

Array Indexing

One-dimensional indexing

```
arr = np.array([10, 20, 30, 40])

print(arr[0])    # 10
print(arr[2])    # 30
print(arr[-1])   # 40
```

Two-dimensional indexing

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matrix[0, 1]) # 2
print(matrix[1, 2]) # 6
```

Indexing convention

Python and NumPy use zero-based indexing.

Array Slicing

One-dimensional slicing

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[1:4])
# [20 30 40]

print(arr[::2])
# [10 30 50]
```

Two-dimensional slicing

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(matrix[0:2, 1:3])
# [[2 3]
#   [5 6]]
```

Views and Copies

Many NumPy slices return a **view** of the original array instead of an independent copy.

```
arr = np.array([10, 20, 30, 40])
```

```
view = arr[1:3]
```

```
view[0] = 999
```

```
print(arr)
```

```
# [ 10 999  30  40]
```

Create an independent copy when needed:

```
copy = arr[1:3].copy()
```

Important

Modifying a view may modify the original array.

Quick Check: Indexing and Slicing

Question 1. What does `arr[0]` return?

- ▶ A. The first element
- ▶ B. The last element
- ▶ C. The array shape
- ▶ D. The array size

Question 2. In a two-dimensional array, `matrix[1, 2]` refers to:

- ▶ A. Row index 1 and column index 2
- ▶ B. Row 1 only
- ▶ C. Column 2 only
- ▶ D. The array dimensions

Question 3. True or false? A NumPy slice may share memory with the original array.

Reshaping Arrays

Reshaping changes array dimensions without changing its data.

```
arr = np.arange(12)
matrix = arr.reshape(3, 4)
print(matrix)
# [[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]]
```

Use -1 to infer a dimension:

```
matrix = arr.reshape(2, -1)
```

Flatten an array:

```
flat = matrix.flatten()
flat_view = matrix.ravel()
```

`flatten()` returns a copy; `ravel()` often returns a view when possible.

Resizing Arrays

`resize()` changes shape and may change the number of elements.

```
arr = np.array([1, 2, 3, 4])
resized = np.resize(arr, (2, 3))
print(resized)
# [[1 2 3]
#   [4 1 2]]
```

Compare

- ▶ `reshape()` requires the total number of elements to remain compatible.
- ▶ `resize()` can create a different total size by repeating values.

Stacking Arrays

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
np.vstack((a, b))  
# [[1 2 3]  
#   [4 5 6]]
```

```
np.hstack((a, b))  
# [1 2 3 4 5 6]
```

```
np.stack((a, b), axis=0)
```

Purpose

Stacking combines arrays into larger arrays along a chosen axis.

Splitting Arrays

Splitting divides arrays into smaller arrays.

```
arr = np.array([1, 2, 3, 4, 5, 6])
parts = np.split(arr, 3)
print(parts)
# [array([1, 2]), array([3, 4]), array([5, 6])]
```

For multidimensional arrays:

```
matrix = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8]
])

left, right = np.hsplit(matrix, 2)
```

Quick Check: Reshaping and Splitting

Question 1. Which method changes an array from one shape to another?

- ▶ A. `reshape()`
- ▶ B. `mean()`
- ▶ C. `split()`
- ▶ D. `sort_index()`

Question 2. What does `-1` mean in `reshape()`?

- ▶ A. NumPy should infer that dimension
- ▶ B. Delete one dimension
- ▶ C. Reverse the array
- ▶ D. Convert values to negative numbers

Question 3. Which function stacks arrays vertically? `np.vstack()`, `np.mean()`, `np.split()`, or `np.random()`?

Broadcasting: Core Idea

Broadcasting allows NumPy to perform operations between arrays of compatible shapes.

```
arr = np.array([1, 2, 3, 4])
result = arr + 10
print(result)
# [11 12 13 14]
```

Basic rule

Starting from the rightmost dimensions, two dimensions are compatible when:

- ▶ they are equal; or
- ▶ one of them is 1.

Broadcasting Between Arrays

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

row = np.array([10, 20, 30])
result = matrix + row
print(result)
# [[11 22 33]
#  [14 25 36]]
```

Incompatible shapes

```
a = np.ones((2, 3))
b = np.ones((2, 2))
# a + b raises a broadcasting error
```

Basic Arithmetic Operations

NumPy supports element-wise arithmetic.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)    # [5 7 9]
print(a - b)    # [-3 -3 -3]
print(a * b)    # [ 4 10 18]
print(a / b)
print(a ** 2)   # [1 4 9]
print(b % a)
```

Important distinction

* performs element-wise multiplication, not matrix multiplication.

Quick Check: Broadcasting and Arithmetic

Question 1. What does broadcasting allow?

- ▶ A. Operations on arrays with compatible different shapes
- ▶ B. Automatic internet transmission
- ▶ C. Conversion of arrays into files
- ▶ D. Removal of all dimensions

Question 2. Two dimensions are broadcasting-compatible when they are equal or:

- ▶ A. One of them is 1
- ▶ B. Both are negative
- ▶ C. Both are text
- ▶ D. Their sum is zero

Question 3. What does `a * b` do for arrays of the same shape?

Aggregation Functions

Aggregation functions summarize an array.

```
arr = np.array([9, 3, 3, 5])
```

```
print(arr.sum())      # 20
print(arr.mean())     # 5.0
print(arr.min())
print(arr.max())
print(np.median(arr))
print(np.var(arr))
print(np.std(arr))
```

Typical use

Aggregation turns raw arrays into summary statistics.

Aggregation Along an Axis

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matrix.sum(axis=0))
# [5 7 9]

print(matrix.sum(axis=1))
# [ 6 15]
```

- ▶ axis=0: aggregate down rows, producing column-wise results.
- ▶ axis=1: aggregate across columns, producing row-wise results.

Universal Functions

A universal function, or **ufunc**, performs element-wise operations on arrays.

```
arr = np.array([1, 4, 9, 16])
print(np.sqrt(arr))
# [1.  2.  3.  4.]

print(np.exp(np.array([0, 1, 2])))
```

```
values = np.array([1, np.e, np.e**2])
print(np.log(values))

angles = np.array([0, np.pi / 2, np.pi])
print(np.sin(angles))
```

Other examples: `np.abs()`, `np.round()`.

Quick Check: Aggregation and Ufuncs

Question 1. Which function calculates the arithmetic mean?

- ▶ A. `np.mean()`
- ▶ B. `np.stack()`
- ▶ C. `np.dot()`
- ▶ D. `np.reshape()`

Question 2. In a two-dimensional array, what does `axis=0` generally aggregate over?

- ▶ A. Rows, producing column-wise results
- ▶ B. Columns, producing row-wise results
- ▶ C. File names
- ▶ D. Data types

Question 3. Which function calculates square roots element by element?

Comparison and Boolean Filtering

NumPy supports element-wise comparisons.

```
arr = np.array([10, 20, 30, 40])
print(arr > 20)
# [False False  True  True]

selected = arr[arr > 20]
print(selected)
# [30 40]
```

Combine conditions:

```
selected = arr[(arr >= 20) & (arr <= 30)]
print(selected)
# [20 30]
```

Use & for AND, | for OR, and ~ for NOT.

Quick Check: Boolean Operations

Question 1. What does `arr[arr > 20]` return?

- ▶ A. Elements greater than 20
- ▶ B. The array size
- ▶ C. The array data type
- ▶ D. All elements converted to Boolean values

Discussion. Why is Boolean filtering useful in data analysis?

Matrix Multiplication

```
A = np.array([
    [1, 2],
    [3, 4]
])

result = np.dot(A, A)
print(result)
# [[ 7 10]
#  [15 22]]

result = A @ A
```

Element-wise vs. matrix multiplication

- ▶ $A * A$: element-wise multiplication.
- ▶ $A @ A$: matrix multiplication.

Matrix Operations

```
print(A.T)

determinant = np.linalg.det(A)
print(determinant)

inverse = np.linalg.inv(A)
print(inverse)
```

```
A = np.array([
    [2, 1],
    [1, 3]
])

b = np.array([8, 13])
x = np.linalg.solve(A, b)
print(x)
```

Condition

An inverse exists only for a square, nonsingular matrix.

Eigenvalues and Vector Products

```
eigenvalues, eigenvectors = np.linalg.  
    eig(A)  
  
print(eigenvalues)  
print(eigenvectors)
```

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
  
print(np.inner(a, b))  
print(np.outer(a, b))  
print(np.dot(a, b))  
print(np.vdot(a, b))
```

For real one-dimensional arrays, `dot` and `vdot` are often the same. `vdot` also handles complex conjugation.

Quick Check: Linear Algebra

Question 1. Which operator performs matrix multiplication?

- ▶ A. @
- ▶ B. %
- ▶ C. //
- ▶ D. &

Question 2. Which function calculates a matrix inverse?

- ▶ A. `np.linalg.inv()`
- ▶ B. `np.mean()`
- ▶ C. `np.resize()`
- ▶ D. `np.random()`

Question 3. Explain the difference between `A * B` and `A @ B`.

Random-Number Generation

For new code, NumPy commonly uses a random-number generator object.

```
rng = np.random.default_rng()
```

```
rng = np.random.default_rng(42)
values = rng.integers(
    low=1,
    high=10,
    size=5
)
print(values)
```

```
values = rng.normal(
    loc=0,
    scale=1,
    size=5
)
print(values)
```

Note

NumPy random-number generators are for numerical and statistical work; they should not be treated as cryptographically secure generators.

Common Distributions

```
rng = np.random.default_rng(42)

rng.uniform(low=0, high=1, size=5)
rng.normal(loc=0, scale=1, size=5)
rng.binomial(n=10, p=0.5, size=5)
rng.poisson(lam=3, size=5)
rng.exponential(scale=2, size=5)
rng.chisquare(df=4, size=5)
```

Reproducibility

Using the same seed helps reproduce the same pseudorandom sequence in a compatible environment.

Statistical Functions

```
data = np.array([12, 15, 18, 20, 25])

print(np.mean(data))
print(np.median(data))
print(np.var(data))
print(np.std(data))
print(np.percentile(data, 25))
print(np.quantile(data, 0.75))
```

Population and sample conventions

By default, `np.var()` and `np.std()` use `ddof=0`. For the common sample convention, use `ddof=1`.

Missing Numerical Values

NumPy commonly represents missing numerical values with `np.nan`.

```
data = np.array([10.0, 20.0, np.nan, 40.0])
print(np.mean(data))
# nan

print(np.nanmean(data))
print(np.nanmedian(data))
print(np.nanstd(data))

print(np.isnan(data))
clean_data = data[~np.isnan(data)]
```

More complete missing-data workflows are commonly handled with Pandas.

Quick Check: Random Numbers and Statistics

Question 1. Which method creates a modern NumPy random-number generator?

- ▶ A. `np.random.default_rng()`
- ▶ B. `np.random.file()`
- ▶ C. `np.create_random_array()`
- ▶ D. `np.random.text()`

Question 2. Why is a seed useful?

- ▶ A. It helps reproduce a pseudorandom sequence
- ▶ B. It makes values truly unpredictable
- ▶ C. It removes every random value
- ▶ D. It converts data into strings

Question 3. Which parameter gives the usual sample standard deviation convention? `ddof=1`, `axis=-100`, `dtype="sample"`, or `copy=False`?

Vectorized Operations

Vectorization applies an operation to an entire array at once.

```
a = np.arange(5)
result = a * 10
print(result)
# [ 0 10 20 30 40]
```

```
a = list(range(5))
result = []
for value in a:
    result.append(value * 10)
print(result)
```

Typical benefits

Shorter code, easier reading, faster operations for large numerical arrays, and better fit for scientific workflows.

Vectorized Conditional Logic

```
arr = np.array([10, 20, 30, 40])

labels = np.where(
    arr >= 30,
    "high",
    "low"
)

print(labels)
# ['low' 'low' 'high' 'high']
```

Interpretation

Vectorized conditions are useful for labeling, screening, filtering, and transforming datasets.

Quick Check: Vectorization

Question 1. What is a major advantage of vectorization?

- ▶ A. It applies array operations without explicit Python loops
- ▶ B. It converts every array into text
- ▶ C. It eliminates the need for memory
- ▶ D. It prevents all errors

Discussion. When might vectorized code be easier to maintain than loop-based code?

Memory Considerations

NumPy arrays usually store homogeneous values in a regular memory layout.

```
arr = np.array([1, 2, 3, 4], dtype=np.int32)
print(arr.nbytes)

small_values = np.array([1, 2, 3, 4], dtype=np.int8)
print(small_values.dtype)
print(small_values.nbytes)
```

Data type choice

A smaller data type may reduce memory use, but it must represent the required value range and precision.

Converting Data Types

```
arr = np.array([1.2, 2.8, 3.5])
integers = arr.astype(np.int32)
print(integers)
# [1 2 3]
```

Important

Conversion from floating-point values to integers removes the fractional part.

Sorting and Searching

```
arr = np.array([9, 3, 7, 1])
sorted_arr = np.sort(arr)
print(sorted_arr)
# [1 3 7 9]
```

```
arr = np.array([10, 20, 30, 40])
indices = np.where(arr > 20)
print(indices)
```

```
arr = np.array([1, 2, 2, 3, 3, 3])
values, counts = np.unique(arr, return_counts=True)
print(values)
print(counts)
```

Working with Images

Digital images can be represented as NumPy arrays.

- ▶ Grayscale image: two-dimensional array.
- ▶ Color image: three-dimensional array containing height, width, and color channels.

```
image = np.array([
    [0, 128, 255],
    [255, 128, 0],
    [50, 100, 150]
])
print(image.shape)
# (3, 3)

brighter = np.clip(image + 30, 0, 255)
```

`np.clip()` keeps values inside a valid range.

Quick Check: Memory, Search, Images

Question 1. Which attribute returns the number of bytes used by array elements?

- ▶ A. `nbytes`
- ▶ B. `ndim`
- ▶ C. `mean`
- ▶ D. `shape`

Question 2. Which function returns unique array values?

- ▶ A. `np.unique()`
- ▶ B. `np.reshape()`
- ▶ C. `np.random()`
- ▶ D. `np.outer()`

Question 3. A grayscale image is commonly represented as what type of array?

Integration with Pandas

A Pandas Series or DataFrame can exchange data with NumPy.

```
import numpy as np
import pandas as pd

arr = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])

df = pd.DataFrame(arr, columns=["A", "B"])
array_from_df = df.to_numpy()

df["A_sqrt"] = np.sqrt(df["A"])
```

Integration with Scikit-learn

Many Scikit-learn models accept NumPy arrays as input.

```
from sklearn.linear_model import LinearRegression

X = np.array([[1], [2], [3], [4]])
y = np.array([2, 4, 6, 8])

model = LinearRegression()
model.fit(X, y)

prediction = model.predict(np.array([[5]]))
print(prediction)
```

NumPy supports feature matrices, target vectors, model predictions, numerical preprocessing, and evaluation calculations.

Common NumPy Errors

Shape mismatch

```
a = np.ones((2, 3))  
b = np.ones((2, 2))  
# a + b raises an error
```

Invalid reshape

```
arr = np.arange(10)  
# arr.reshape(3, 4) raises an error
```

Division by zero

```
arr = np.array([1.0, 0.0])  
print(1 / arr)
```

Integer overflow

```
arr = np.array([127], dtype=np.int8)  
print(arr + 1)
```

Good Practices

- ▶ Import NumPy using `import numpy as np`.
- ▶ Use vectorized operations instead of Python loops when practical.
- ▶ Check `shape`, `ndim`, and `dtype` before complex operations.
- ▶ Use broadcasting only when intended shape behavior is clear.
- ▶ Distinguish element-wise multiplication from matrix multiplication.
- ▶ Use `.copy()` when an independent array is required.
- ▶ Choose data types carefully to balance range, precision, and memory.
- ▶ Use a fixed random seed when reproducibility is important.
- ▶ Treat NaN, infinity, and overflow explicitly.

Quick Check: Integration and Errors

Question 1. Which Pandas method converts a DataFrame to a NumPy array?

- ▶ A. `to_numpy()`
- ▶ B. `to_list_only()`
- ▶ C. `as_matrix_text()`
- ▶ D. `convert_numpy_file()`

Question 2. In machine learning, a two-dimensional NumPy array commonly represents:

- ▶ A. A feature matrix
- ▶ B. A filename
- ▶ C. A chart title
- ▶ D. A package installer

Question 3. Why does `np.arange(10).reshape(3, 4)` fail?

Content Summary

Topic	Main idea
NumPy	Core Python library for numerical computing.
ndarray	N-dimensional homogeneous array.
Vectorization	Array operations without explicit Python loops.
Broadcasting	Operations on compatible different shapes.
Indexing and slicing	Access and extract array elements.
Reshaping	Change array dimensions.
Aggregation	Sums, means, minima, maxima, variance, and standard deviation.
Universal functions	Fast element-wise mathematical functions.
Linear algebra	Matrix multiplication, inverse, determinant, and eigenvalues.
Random generation	Generate values from probability distributions.
Statistics	Mean, median, variance, standard deviation, and percentiles.
Integration	Works closely with Pandas, SciPy, and Scikit-learn.

Practice: Array Creation

Exercise 1

- 1 Create an array containing values from 1 to 20.
- 2 Reshape it into a 4×5 matrix.
- 3 Print its shape, number of dimensions, size, and data type.

Exercise 2

Using a 4×5 matrix:

- 1 Extract the first row.
- 2 Extract the last column.
- 3 Extract the central 2×3 section.
- 4 Select all even values using Boolean filtering.

Practice: Statistics and Broadcasting

Exercise 3: Statistics

Create an array of ten numerical values and calculate mean, median, variance, sample standard deviation, minimum, maximum, and the 25th, 50th, and 75th percentiles.

Exercise 4: Broadcasting

- 1 Create a 3×4 matrix.
- 2 Create a one-dimensional array with four values.
- 3 Add the one-dimensional array to each row of the matrix.
- 4 Explain why broadcasting is valid.

Practice: Linear Algebra and Missing Values

Exercise 5: Linear algebra

Given

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 8 \\ 13 \end{bmatrix},$$

calculate the determinant of A , inverse of A , solve $Ax = b$, and verify with $A @ x$.

Exercise 6: Missing values

Given `[10.0, np.nan, 20.0, 30.0, np.nan, 40.0]`, count missing values, calculate the mean while ignoring missing values, remove missing values, and replace missing values with the median.

Final Reflection

- ▶ Which NumPy topic is most important for data analysis workflows?
- ▶ When should you check `shape` before running an operation?
- ▶ How do vectorization and broadcasting reduce manual loops?
- ▶ Why is it important to distinguish views from copies?
- ▶ When should you use `ddof=1` instead of the default?
- ▶ What common NumPy error are you most likely to encounter in practice?